clone the repo

git clone git@github.com:mankings/workshop-docker.git

Docker 101

workshop

Miguel Matos

What is?

The Problem

any code that runs perfectly fine on your computer

will fail to run on somebody else’s

- underlying operating system/infrastructure
- missing or outdated dependencies
- environment variables

The Problem

any code that runs perfectly fine on your computer

will fail to run on somebody else’s

- underlying operating system/infrastructure
- missing or outdated dependencies
- environment variables

The Evolution of Deployment

Bare Metal

high costs
no isolation

Virtual Machines

shared hardware
full isolation
resource-heavy

Containers

efficient and isolated
lightweight
portable

The Solution in a nutshell

source

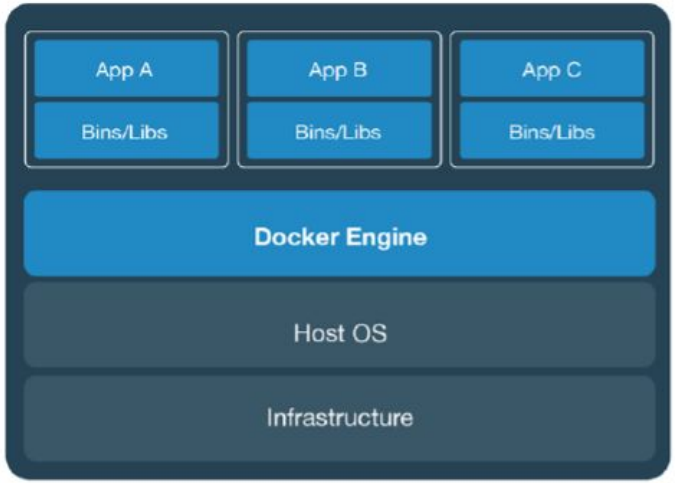
Containers

Container

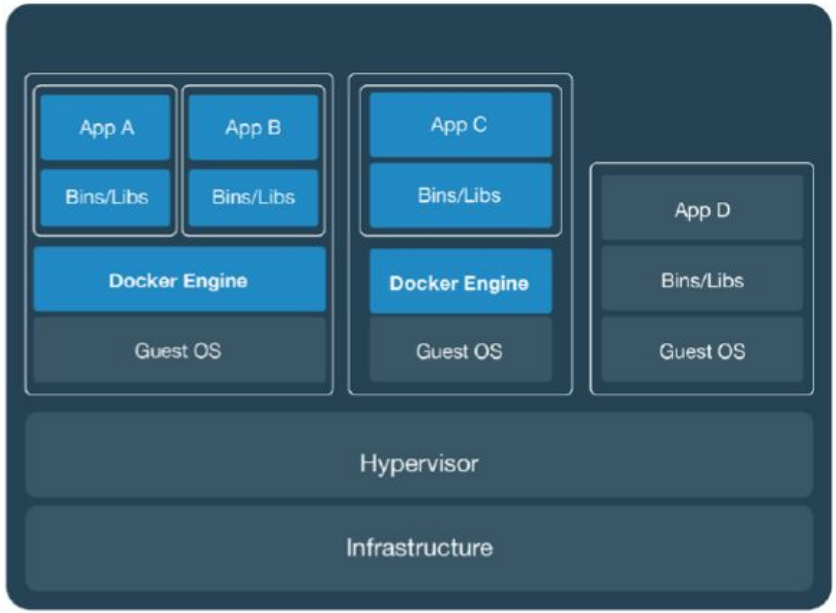
- Lightweight, executable units of software that contain everything needed to run an application.

Docker is a platform designed to make it easier to create, deploy, and run applications by using containers.

Containers vs VMs



Containers



Virtual Machines

Why containers

Consistency

- "Works on my machine" issue resolved.

Isolation

- Applications run in separate containers without affecting each other.

Portability

- Run anywhere: on-premises, cloud, or hybrid environments.

Efficiency

- Lightweight compared to traditional VMs, uses fewer resources.

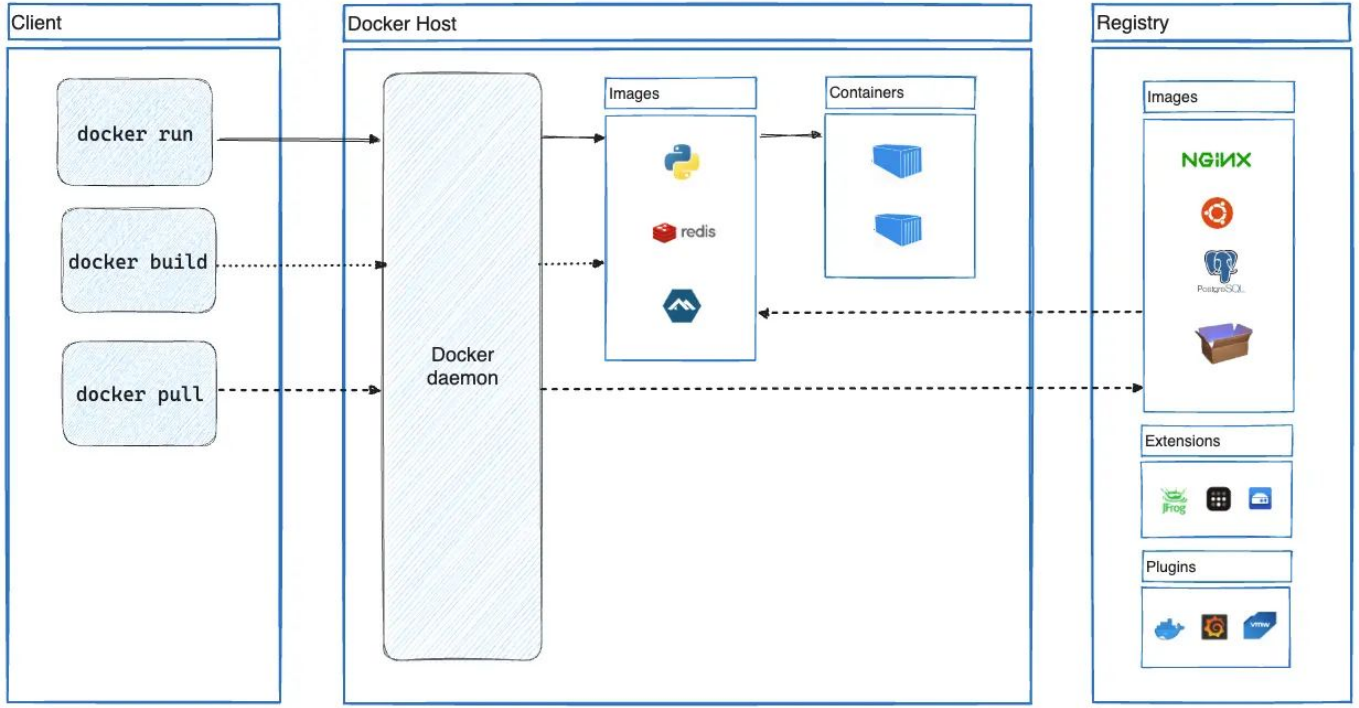
Docker real-world uses

- Netflix
- Spotify
- Pinterest
- The New York Times
- …

- IES
- TQS
- CBD
- SIO
- TPW
- …

How does it work?

Docker Architecture



Containers and Images

container

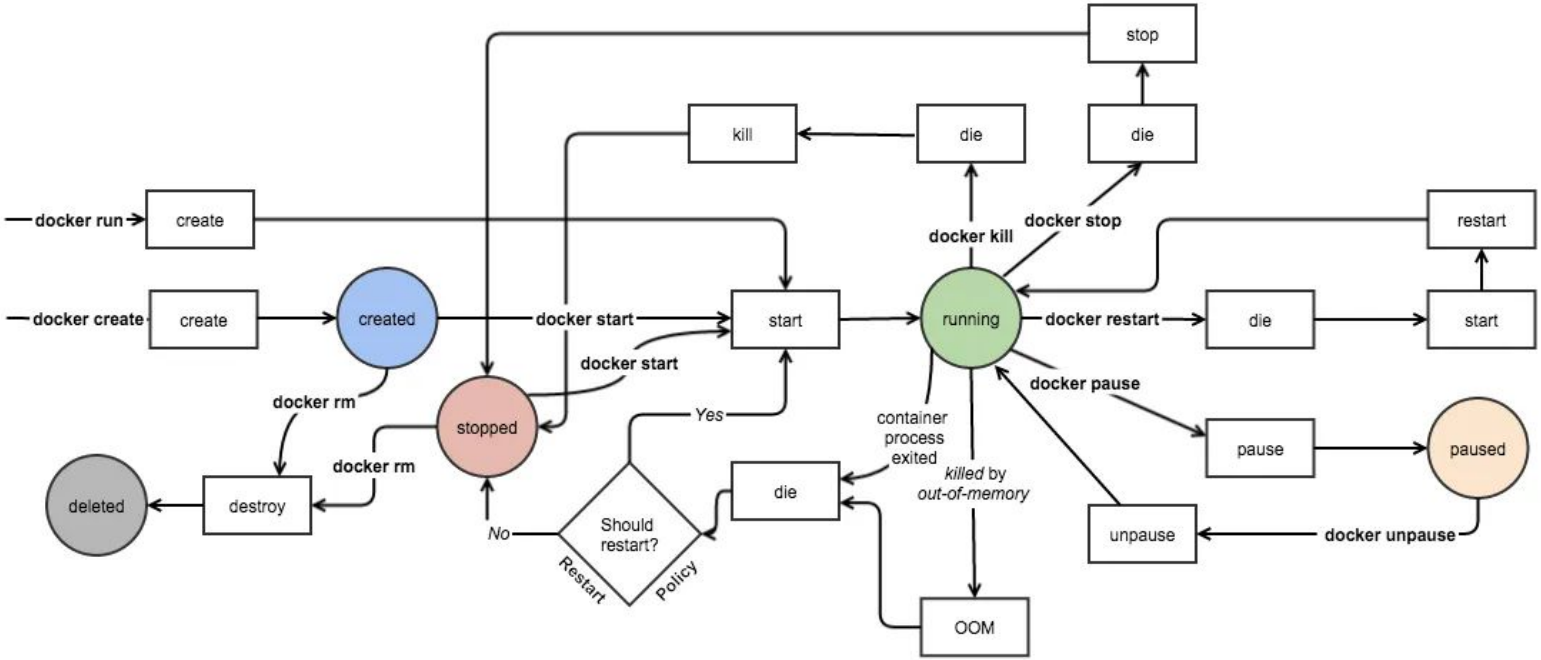
image

Dockerfile

1. Base image
2. Modifications
3. Start command

```
# Base image
FROM python:3.12-alpine (last pushed 3 days ago)

# Image labels/tags
LABEL maintainer="Miguel Matos"

# Execute the command to create a folder for the application code
RUN mkdir /app

# Define the working directory inside the Docker container
WORKDIR /app

# Copy external files into the working directory
# In this case, copying python requirements file
COPY requirements.txt requirements.txt

# Install dependencies with pip
RUN pip install -r requirements.txt

# Copy the external code into the working directory
COPY app.py app.py

# Expose the port of the service
EXPOSE 8080

# Define the command to run when the container starts
CMD ["python3", "app.py"]
```

Container Lifecycle



Docker Basics

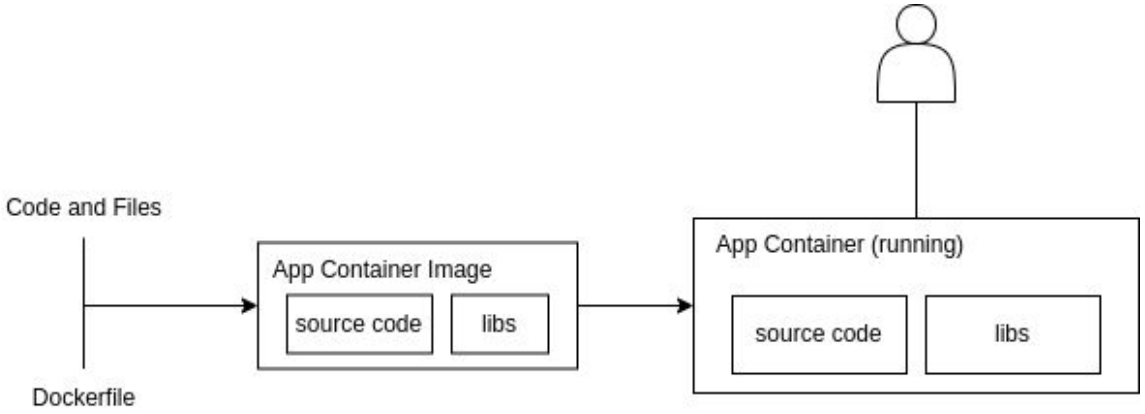
Basic Commands

- docker pull - pull an image from a registry
- docker run - run a container
- docker ps - list running containers
- docker stop - stop a container
- docker rm - remove/delete a container
- docker build - build an image
- docker rmi - remove an image
- docker exec - execute a command inside a container
- docker logs - check or follow the logs of a container

Hands-on!

Example 1 - Simple Application

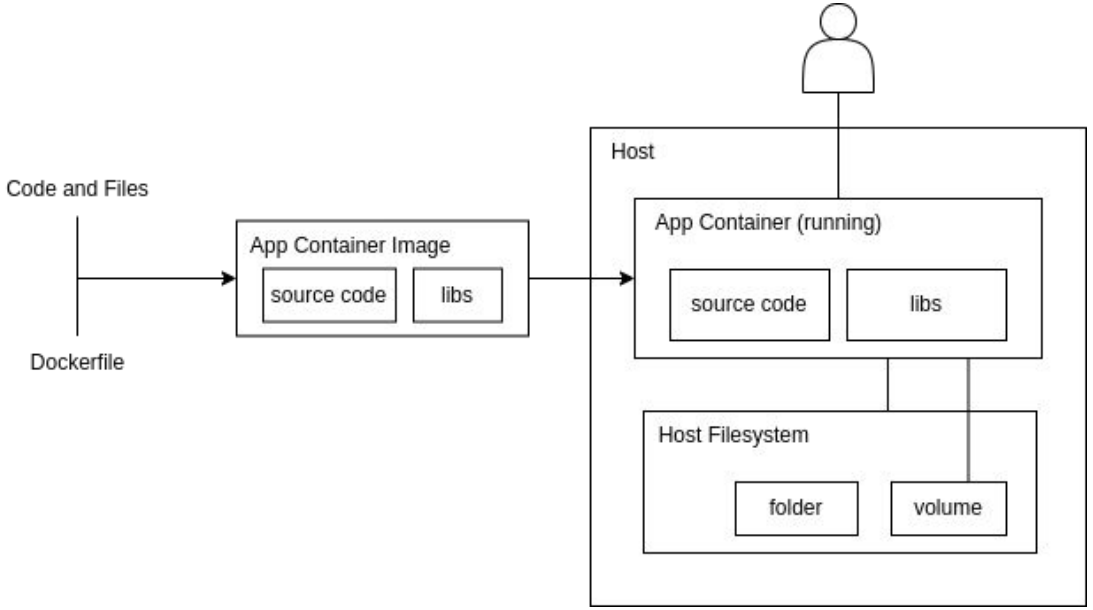
- *app.py* - application source code
- *requirements.txt* - required libraries/packages
- *Dockerfile* - instructions to build the container image



Example 2 - Persistence

Containers are ephemeral - data within them is lost upon stop or destruction

- Volumes
- Bind mounts



Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications.

Uses .yaml configuration files, defining the following:

- Services
- Networking
- Volumes
- Configuration

docker-compose.yml file

```yaml
services:
    app:
        build:
            context: .
            dockerfile: Dockerfile.app
        volumes:
            - ./www:/app/www

    nginx:
        build:
            context: .
            dockerfile: Dockerfile.nginx
        ports:
            - 8081:80
```

Docker Compose basics

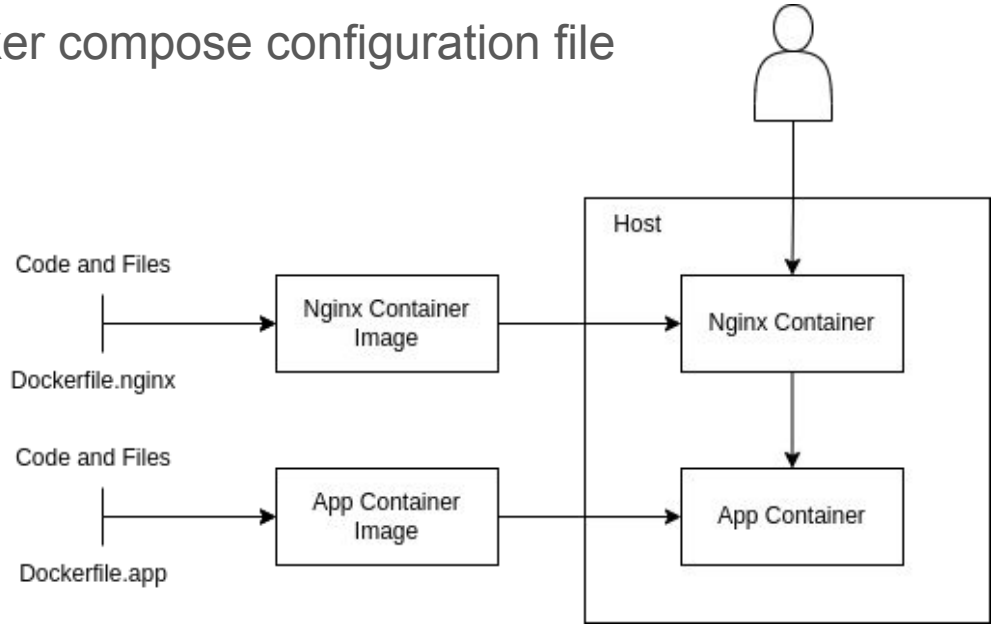
Basic Commands

- docker compose up [-d] - launch services
- docker compose down - stop services

- docker compose ps - list services
- docker compose pull - pull the image of a service
- docker compose build - build service images
- …

Example 3 - Multi-container Application

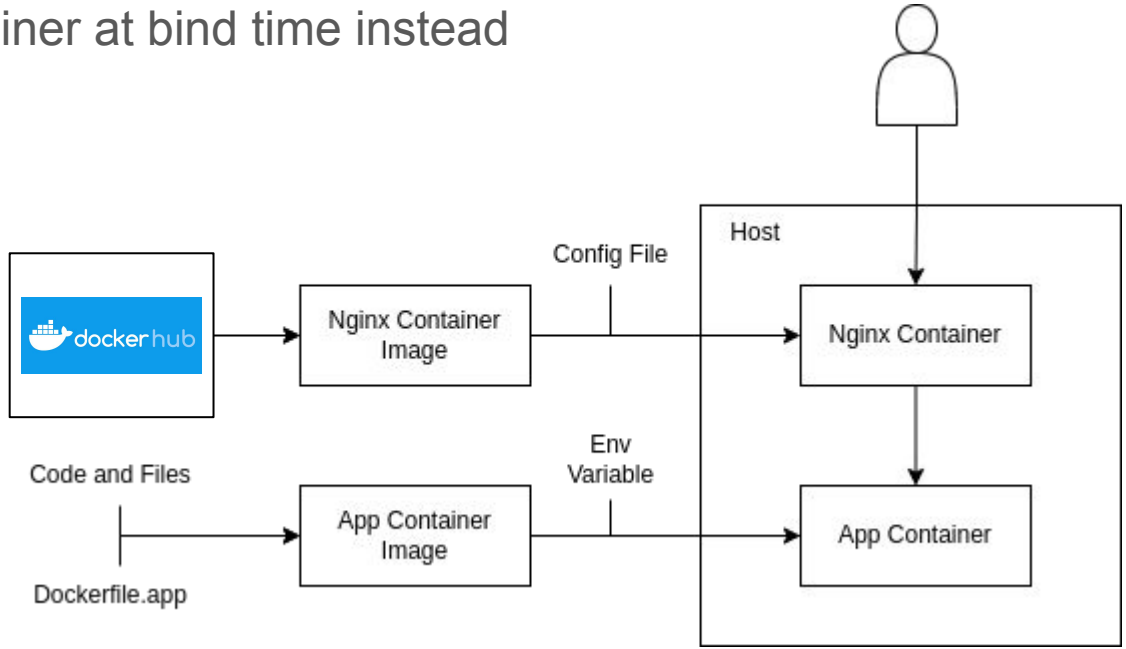
- *Dockerfile.app* - main application Dockerfile
- *Dockerfile.nginx* - Nginx service Dockerfile
- *docker-compose.yml* - docker compose configuration file



Example 4 - Configs

Used to manage data required by the application at runtime.

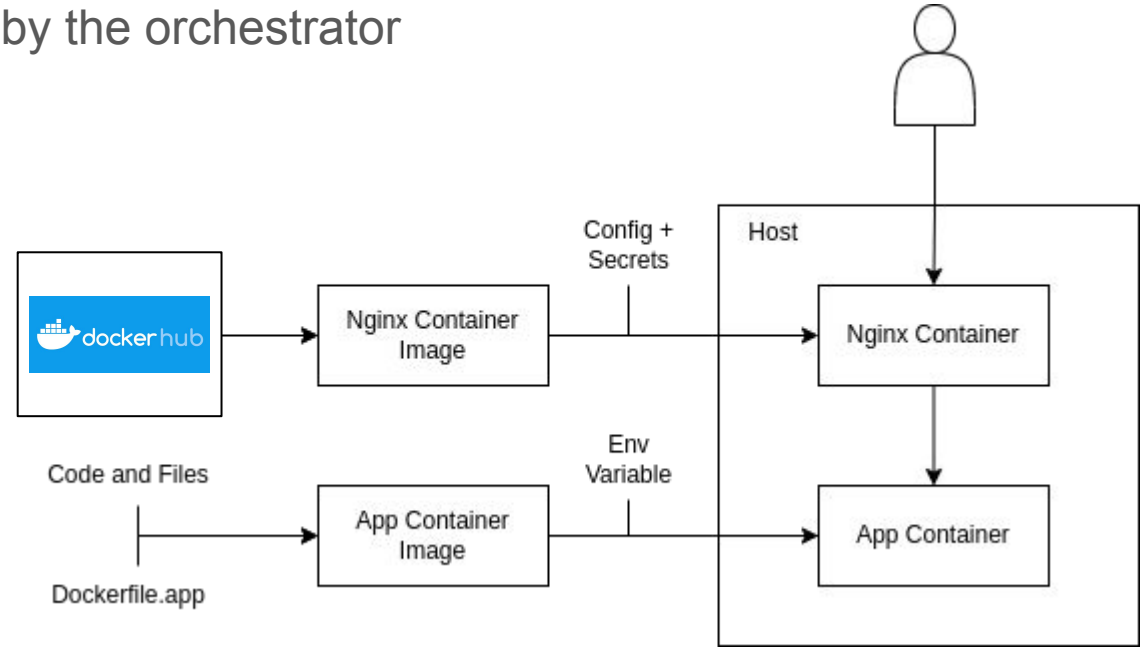
Add configuration to container at bind time instead of build time.



Example 5 - Secrets

Used to manage sensitive data required by the application at runtime.

Controlled and encrypted by the orchestrator



Going Further

- Environment Management
  - Consistency between *dev*, *staging* and *prod* environments
- CI/CD Pipelines
  - Automated pipelines for building, testing, and pushing application artefacts
- Orchestration
  - Kubernetes or Docker Swarm to manage lots of containers and application scaling

Conclusion

- Docker simplifies the process of building, sharing, and running applications by packaging them into portable units (containerization!)

Key Benefits:

- Consistency
- Portability
- Efficiency

Thanks!

questions?

and remember to cleanup your old images :)